

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I Challa Priyadarshini(19231111)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Ch.Priyadarshini

CRYPTOGRAPHY AND NETWORK SECURITY

CASE STUDY ANALYSIS AND SECURE SOLUTIONS

Introduction

This assignment analyzes five practical cybersecurity scenarios. Each case identifies the cryptographic weakness, explains the security impact, and proposes a standards-based improvement.

1. MD5 Used for Password Storage

Problem Analysis

MD5 is unsuitable for storing passwords. It is a very fast general-purpose hash, and its collision resistance is broken. More importantly for password storage, its speed allows attackers to test huge numbers of password guesses after stealing a database. If passwords were also stored without unique salts, identical passwords produce identical hashes and precomputed attacks become easier.

Weaknesses

- MD5 is obsolete for security-sensitive collision resistance.
- MD5 is extremely fast, which is undesirable for password hashing.
- Unsalted MD5 causes identical passwords to have identical database values.
- Fast hashing makes dictionary and brute-force attacks much more practical.
- MD5 does not provide password-specific memory or computational cost controls.

Recommended Secure Approach

The preferred modern choice for password storage is a password-specific function such as Argon2id. If a SHA-based technique is specifically required, use PBKDF2-HMAC-SHA-256 or PBKDF2-HMAC-SHA-512 with a unique random salt and a suitably high iteration count.

1. Generate a cryptographically secure random salt for every password.
2. Use PBKDF2-HMAC-SHA-256 with an organization-approved work factor, or preferably Argon2id.
3. Store the salt, algorithm and parameters with the derived password value.
4. Use constant-time comparison during verification.
5. Rate-limit login attempts and add MFA where appropriate.
6. Migrate legacy MD5 records after successful authentication instead of retaining MD5 permanently.

Example SHA-Based Implementation

```
import hashlib, secrets, hmac
```

```
password = b"MyStrongPassword"
```

```
salt = secrets.token_bytes(16)
```

```
iterations = 600_000
```

```
stored = hashlib.pbkdf2_hmac(
```

```

"sha256", password, salt, iterations
)

test = hashlib.pbkdf2_hmac(
    "sha256", b"MyStrongPassword", salt, iterations
)

print("Salt:", salt.hex())
print("Password valid:", hmac.compare_digest(stored, test))

```

Comparison

Property	MD5	PBKDF2-HMAC-SHA-256 / Argon2id
Password-specific	No	Yes
Speed	Very fast	Intentionally slower
Unique salt	Must be added separately	Required
Password-guessing resistance	Poor	Much stronger when configured correctly
Recommended today	No	Yes

Conclusion

The breach was enabled by an unsuitable password-storage design. MD5 should be replaced with Argon2id where possible; when a SHA-based solution is required, PBKDF2-HMAC-SHA-256 with unique salts and a strong work factor is a safer approach.

2. Unauthorized Banking Transactions Despite Digital Signatures

Problem Analysis

A valid digital signature should make unauthorized modification of the signed data detectable. If unauthorized transactions are approved, the problem may be in what is signed, how the key is managed, how identity is validated, or how replay and transaction context are checked.

Possible Flaws

- Only a transaction ID or partial data is signed instead of the complete transaction.
- Amount, destination, currency, timestamp, nonce or transaction type is not covered by the signature.
- The public key is not correctly bound to the authorized customer.
- Private signing keys are exposed, shared, or poorly protected.
- Expired or revoked keys/certificates are accepted.
- Weak or obsolete signature padding/algorithms are used.
- A previously valid signature can be replayed because freshness is not enforced.
- The UI displays transaction data different from the canonical data actually signed.

Recommended Improvements

7. Create one canonical representation containing every security-critical transaction field.

8. Sign the complete canonical transaction, including a unique transaction ID/nonce and suitable freshness information.
9. For RSA, use RSA-PSS rather than textbook RSA or insecure legacy constructions.
10. Use modern standardized signature schemes and vetted cryptographic libraries.
11. Validate certificate/public-key identity, validity and key usage.
12. Protect private keys using secure key-management or hardware-backed storage where appropriate.
13. Reject reused transaction IDs and stale signatures.
14. Require step-up authentication for high-risk transactions.

Secure Transaction Flow

Display exact transaction → canonicalize all fields → sign complete transaction → verify trusted public key → verify signature → check nonce/freshness → check authorization and policy → execute.

Conclusion

Digital signature mathematics alone is not sufficient. The signature must be bound to the exact transaction and the correct identity, and the private key must be protected. RSA-PSS or another standardized modern signature scheme should be used with strong key management and anti-replay controls.

3. Kerberos Replay Attack

How the Attack Can Occur

A replay attack occurs when an attacker captures a previously valid authentication exchange and sends it again later. If freshness checks are weak or absent, the service may accept the captured credentials as a new authentication request.

Possible Causes

- Authenticator timestamps are not checked correctly.
- Replay-cache protection is disabled or incorrectly implemented.
- Clock synchronization is poor, causing administrators to weaken freshness checks.
- Tickets or authenticators remain valid longer than necessary.
- A custom Kerberos integration incorrectly validates tickets.
- Credentials or session keys are exposed on compromised endpoints.

Enhancements

15. Use a standard Kerberos implementation rather than a custom authentication protocol.
16. Require and validate authenticators with timestamps.
17. Maintain reliable time synchronization and enforce an acceptable clock-skew window.
18. Use replay caches where appropriate to reject reused authenticators.
19. Use suitable ticket lifetimes and renewal policies.
20. Protect KDC keys, service-account keys and session credentials.
21. Disable obsolete encryption types and monitor abnormal authentication activity.

Replay Protection Controls

Control	Purpose
---------	---------

Timestamp/authenticator	Provides freshness
Replay cache	Detects reused authenticators
Clock synchronization	Makes freshness checks reliable
Short ticket lifetime	Limits the attack window
Protected KDC keys	Protects the trusted authentication authority

Conclusion

Kerberos is designed with replay resistance, but misconfiguration or insecure custom integration can defeat it. Correct timestamp validation, replay-cache use, synchronized clocks, appropriate ticket lifetimes and secure key management are essential.

4. Simple Hash Integrity Check vs HMAC

Why the Simple Hash Fails

If a sender sends M together with $\text{SHA-256}(M)$, an attacker who changes M to M' can simply calculate $\text{SHA-256}(M')$ and replace the old hash. The receiver sees a matching hash but has no evidence that the message came from the legitimate sender. A plain hash provides integrity checking but not authentication.

Attack Sequence

22. Sender calculates $H = \text{SHA-256}(M)$.
23. Attacker intercepts M and H .
24. Attacker changes M to M' .
25. Attacker calculates $H' = \text{SHA-256}(M')$.
26. Receiver accepts M' because H' matches.

HMAC Solution

HMAC combines a cryptographic hash with a secret shared key. An attacker can observe the message and MAC but cannot normally create a valid MAC for modified data without the secret key.

```
import hashlib, hmac

key = b"shared-secret"
message = b"Transfer 500 to Account123"

mac = hmac.new(key, message, hashlib.sha256).hexdigest()

received = b"Transfer 500 to Account123"
check = hmac.new(key, received, hashlib.sha256).hexdigest()

print("Valid:", hmac.compare_digest(check, mac))

tampered = b"Transfer 5000 to Account123"
tampered_check = hmac.new(key, tampered, hashlib.sha256).hexdigest()
```

```
print("Tampered valid:",  
      hmac.compare_digest(tampered_check, mac))
```

Comparison

Property	Simple SHA-256	HMAC-SHA-256
Detects changes	Yes	Yes
Authenticates shared-key sender	No	Yes
Attacker can recalculate after editing	Yes	Not without key
Secret key required	No	Yes
Non-repudiation	No	No

Conclusion

The simple hash failed because it was not keyed. HMAC-SHA-256 provides both integrity and shared-secret authentication. The secret key must be protected, and verification should use a constant-time comparison such as `hmac.compare_digest()`.

5. X.509 Certificate Trust Validation Errors

Problem Analysis

X.509 authentication depends on a valid certificate chain and correct identity checks. A typical chain is Root CA → Intermediate CA → Server Certificate. The client must be able to build a valid path to a trusted root and verify certificate signatures, validity dates, hostname, key usage and applicable revocation information.

Possible Issues

- Expired or not-yet-valid certificate.
- Hostname does not match the Subject Alternative Name (SAN).
- Missing intermediate CA certificate on the server.
- Root CA is not trusted by the client.
- Incorrect or untrusted certificate authority.
- Revoked certificate or failed revocation checking.
- Incorrect server certificate or incomplete chain.
- Incorrect system clock.
- Key Usage or Extended Key Usage does not permit the intended operation.
- Certificate and private key do not match after renewal.

Recommended Solutions

27. Maintain an inventory of certificates and their owners, hosts and expiry dates.
28. Install the complete required intermediate chain on servers.
29. Use correct SAN entries for every hostname.
30. Automate renewal before expiration.
31. Keep trusted root stores current and governed by policy.
32. Verify certificate/private-key pairing during deployment.

33. Maintain accurate system time.
34. Use appropriate revocation mechanisms such as CRL or OCSP when required.
35. Validate Key Usage and Extended Key Usage.
36. Test certificate chains before production deployment and monitor validation failures.

Certificate Validation Checklist

Check	Requirement
Validity	Current time is within certificate validity period
Hostname	Requested hostname matches SAN
Issuer	Expected CA issued the certificate
Chain	Valid path to trusted root exists
Signature	Certificate signatures verify
Key usage	Certificate permits intended purpose
Revocation	Status checked where applicable
Private key	Correct key matches certificate
System time	Client and server clocks are accurate

Conclusion

Frequent X.509 errors are usually certificate lifecycle or trust-chain management problems. Correct SANs, complete intermediate chains, trusted roots, automated renewal, accurate time, private-key protection and appropriate revocation checks provide reliable certificate authentication.

Overall Comparative Summary

Case	Main Weakness	Recommended Solution
MD5 passwords	Fast, obsolete password hashing	Argon2id; PBKDF2-HMAC-SHA-256 if SHA-based method required
Banking signatures	Incomplete signing, key compromise or replay	RSA-PSS/modern standard + full transaction binding + secure key management
Kerberos replay	Weak freshness or replay validation	Authenticators, timestamps, replay cache, short lifetimes
Simple hash	No authentication key	HMAC-SHA-256
X.509	Chain, expiry, hostname or trust-store errors	Complete chain + trusted roots + SAN + lifecycle automation

Final Conclusion

These cases demonstrate that cryptography must be selected and implemented according to the security goal. Passwords need slow, salted password hashing; digital signatures need standardized algorithms and complete transaction binding; Kerberos needs freshness and replay controls; HMAC is required when a shared secret must authenticate messages; and X.509 requires disciplined certificate lifecycle and trust-chain management. Production systems should use current standards and well-tested cryptographic libraries rather than custom cryptographic implementations.